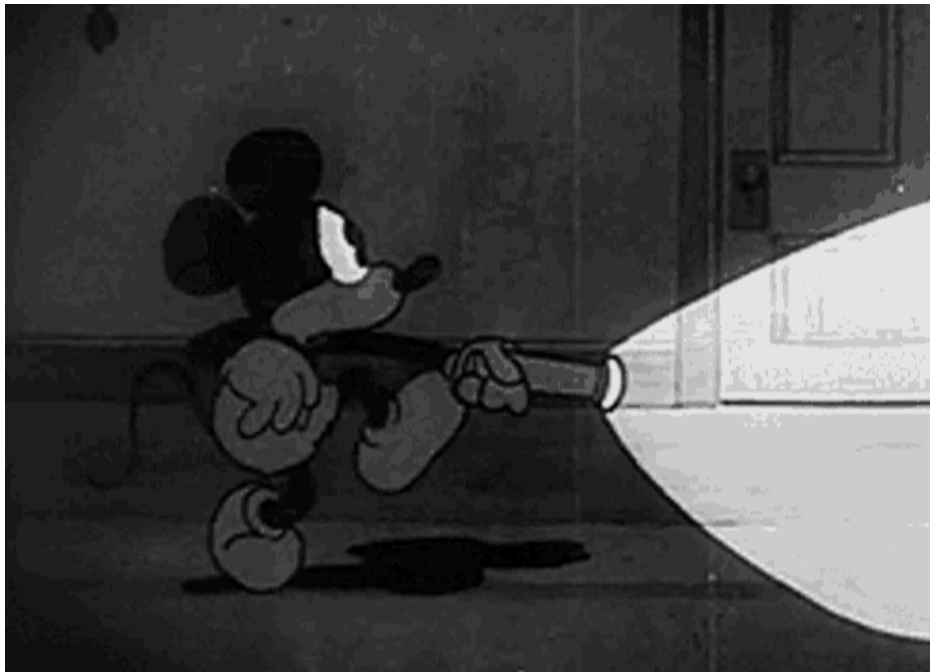


Lab

Blind Adversary

1 Introduction

Hide and Seek Arena aims to help students understand, implement, and optimize search algorithms through an automated arena that simulates the game of hide and seek, where student teams can compete against one another for ranking.



In this assignment, we introduce partial observability. Agents do not have access to the full map at each step; instead, each agent must plan and act using only a local field of view.

Each student team will develop both of the following agents:

- Hide agent: Hide for as long as possible under limited vision.
- Seek agent: Find and catch as quickly as possible under limited vision.

The agents will be automatically matched in the Arena to compute win rates and rankings.

2 Arena

The Arena is an automated framework, implemented in Python, that serves as the competitive environment for the agents. Given the teams' submissions, the Arena will automatically import each team's agents, run a series of matches between teams, and output the results.

2.1 Overview

The model of the Arena is illustrated as below:

Algorithm 1 Agent Environment Interaction Loop

```
1: while episode not terminated do
2:    $a_p \leftarrow \text{PacmanAgent.step}()$ 
3:    $a_g \leftarrow \text{GhostAgent.step}()$ 
4:    $\text{Environment.step}(a_p, a_g)$ 
5:   Update agent positions
6:   Check collisions or termination
7:    $s \leftarrow$  observe new environment state with partial observability
8: end while
```

At each step in Algorithm 1, the provided information includes:

- `map_state`: A partial 21×21 map observation representing the local field of view.
- `my_position`: The agent's current position with coordinates `x` and `y`.
- `enemy_position`: The opponent's position if it is visible, otherwise `None`.
- `step_number`: The current step count.

2.2 Partial Observability and Local View Representation

To construct the map state mentioned above, the Arena restricts what the agent can perceive based on its current coordinates `x` and `y`. Rather than seeing the entire board, the agent is limited to a cross shaped local field of view. This grants the agent sight of its current cell along with up to 5 cells outward in the four cardinal directions being Up, Down, Left, and Right. These four vision rays represent the agent line of sight, but they are strictly blocked by any non traversable

```

# # # # # # # # # # # # # # # # # # # # #
# . . . . . . . . . # . . . . . . . . . #
# . # # # . # # # . # . # # # . # # # . #
# . # # # . # # # . # . # # # . # # # . #
# . . . . . . . . . . . . . . . . . . . #
# . # # # . # . # # # # # . # . # # # . #
# . . . . . # . . . # . . . # . . . . . #
# # # # # . # # # . # . # # # . # # # # #
# # # # # . # . . . G . . . # . # # # # #
# # # # # . # . # # . # # . # . # # # # #
# . . . . . . . . . # . . . # . . . . . #
# # # # # . # . # # # # # . # . # # # # #
# # # # # . # . . . . . . . # . # # # # #
# # # # # . # . # # # # # . # . # # # # #
# . . . . . . . . . # . . . . . . . . . #
# . # # # . # # # . # . # # # . # # # . #
# . . . # . . . . . P . . . . . # . . . #
# # # . # . # . # # # # # . # . # . # # #
# . . . . . # . . . # . . . # . . . . . #
# . # # # # # # # . # . # # # # # # # . #
# . . . . . . . . . . . . . . . . . . . #
# # # # # # # # # # # # # # # # # # # # #

```

Figure 1: Arena map with P for Pacman, G for Ghost, # for Walls, and . for Empty path

obstacles. The moment a wall is encountered, visibility in that specific direction stops completely, hiding all cells behind the wall.

The Arena packages this limited vision into a masked 21×21 grid so the agent can programmatically process what is known and what remains hidden. Within this grid, wall cells are always visible and are marked as 1. The empty paths that successfully fall within the agent line of sight are marked as 0. Meanwhile, any empty space that lies outside this observable region is encoded as -1 to indicate unknown territory. When designing the planning logic, student teams must ensure their agents treat all -1 cells as completely unknown and never blindly assume they are safe to traverse. Similar to the non-blind mode, both agents act simultaneously but may receive different observations due to their positions and limited visibility.

The win conditions are defined as follows:

- The Seek agent wins: if the Seek agent touches the Hide agent with a Manhattan distance strictly less than 2.
- The Hide agent wins: if the maximum number of allowed steps is reached.

Please note that the framework is provided to you for reference, evaluating your own agent against the above conditions may require some tweakings.

The source code of this framework is provided as `pacman.zip` in the [Google Drive folder](#). The source code will be provided to students with the following directory structure:

```
Arena/
├── src/ .....Framework source code do NOT modify
├── submissions/ .....Student workspace
│   ├── example_student/
│   │   ├── agent.py .....Reference implementation
│   │   └── student_id/
│   │       └── agent.py .....Student team agent
│   └── STUDENT_GUIDE.md ..... Student instructions
└── run_game.sh .....Quick run script
```

Besides that, the given folder also includes an **Blind** folder featuring selected instances of previous courses where the **top** submissions are sampled. About the speed, if both agents move at the same speed, the Seek Agent can never win because it cannot make contact with the Hide Agent. To ensure fairness, the Seek Agent moves at a speed of 2 cells per step when moving in a straight line but cannot move with an L shape immediately during a turn.

At each step, a student team's agent must return an action in the following format:

```

1 class AgentInterface(ABC):
2     @abstractmethod
3     def step(self, map_state, my_position, enemy_position, step_number):
4         # Planning under partial observability
5         # For Pacman Agent: return a Move or a tuple of Move and steps, where:
6         # steps is an int from 1 to the configured max straight line speed.
7         # For Ghost Agent: return a Move such as UP, DOWN, LEFT, RIGHT, or STAY

```

The recommended algorithms are: Minimax, Monte Carlo, Alpha beta Pruning, Expectiminimax, etc. adapted for partial observability. Student teams must ensure that their agent returns an action within a maximum of 1 second.

3 Tournament

Each team will participate in both roles, Hide and Seek, and will compete against the agents of all other teams. Based on the outcomes, the win rates are computed as:

$$\text{win_rate}_{\text{hide}} = \frac{\text{Hide wins}}{\text{Hide games}} \quad \text{win_rate}_{\text{seek}} = \frac{\text{Seek wins}}{\text{Seek games}}$$

The results are recorded in the following example table.

Table 1: Match Results Between Teams for Hide and Seek roles

Hide \ Seek	Team 1	Team 2	Team 3	Team 4
Team 1	–	0	1	1
Team 2	0	–	1	0
Team 3	1	0	–	1
Team 4	0	0	1	–

Here, a value of 0 denotes a Hide win, while a value of 1 denotes a Seek win.

Based on Table 1, an example of the win rates for Team 1 is as follows:

- As the Hide agent, Team 1 wins 1 out of 3 matches, resulting in $\text{win_rate}_{\text{hide}} = 33.3\%$.
- As the Seek agent, Team 1 wins 2 out of 3 matches, resulting in $\text{win_rate}_{\text{seek}} = 66.6\%$.

Note that, besides raw winning result, we also record your agent's average steps required to end each match. For Pacman, you should also minimize this value while for Ghost, you should

also maximize this value. In order to tie breaking the teams with similar winning rate, we evaluate their differences between Pacman's and Ghost's average steps where one with lower difference ranks higher.

The scoring criteria for each team is defined as follows:

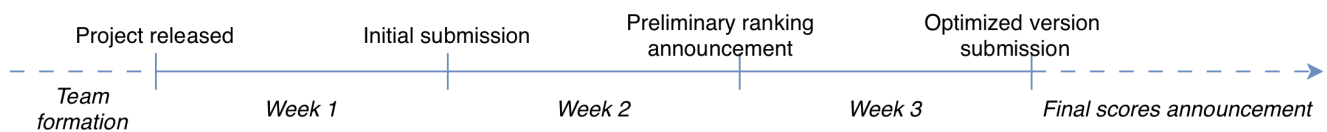
Criterion	Score
Algorithm implementation completeness	3
Ranking in the initial submission as detailed in the timeline in Section 4	Up to 3
Ranking in the optimized submission as detailed in the timeline in Section 4	Up to 4

4 Submission

Each team must submit a single `group_id.zip` file to Moodle containing the following directory structure where `group_id` denotes a single, no zero-leading, integer for your group's ID only, for example please use `1.zip` INSTEAD of `group_1.zip` or `group_01.zip`. The general structure of your submitted zip folder should be:

```
group_id/
├── agent.py
└── etc.
```

The submitted agent is allowed to import other files located within the same submission directory. The allowed extension for storing ML models' weights are: `.onnx`, `.pt/.pth`, `.npy/.npz`. The submission timeline is specified as follows:



5 Notices

Please pay attention to the following notices:

- This is a **GROUP** assignment.
- Late submissions will **not** be accepted under any circumstances.
- This project has two submission phases. Only teams that submitted in the first phase are allowed to submit in the second phase.

- Student teams must ensure that their agent returns an action within a maximum of **1 second** while consuming at most **128 MB** under the setting of Google Colab's CPU-only instance. The built-in `time` library may be used together with all built-in libraries bundled with Python 3. External libraries available for your team includes: numpy, pandas, scipy, gurobi, pytorch, and scikit-learn, together with all of their dependencies in an environment of Python 3.11.
- AI tools are **NOT restricted**; however, students should use them wisely. Lab instructors have the right to conduct additional oral interviews to assess their knowledge of the project.
- Any form of plagiarism, dishonesty, or misconduct will result in a grade of zero for the course.

The end.